

# 让 Agent 管理现代应用栈

SupportOps · Agentist cohort-1

## 从代码到上线，再到持续运行

180 分钟课堂讲义 · 2026-09-12

Vercel · Supabase · Render · Cloudflare · GitHub Actions · Lambda/vLLM · Daytona

贯穿案例：用户提交客服工单，后台 Agent 查询资料并保存回复草稿。Code Agent 编写测试、创建 PR、修复 CI 失败，再验证部署。

本讲义区分已实现代码、部署模板与待实测操作。最新资源及验证状态见同目录 STATUS.md。

# 课程地图

上半场把应用部署上去；下半场让测试、发布、推理与预算形成可验证闭环。

01 从一张工单看完整系统 / 00 – 10 · 10 分钟

02 让 Code Agent 获得平台能力 / 10 – 25 · 15 分钟

03 GitHub 与 Supabase：先让数据有家 / 25 – 45 · 20 分钟

04 Vercel：从仓库到可访问应用 / 45 – 60 · 15 分钟

05 Cloudflare 与支付边界 / 60 – 80 · 20 分钟

06 Render 与 Daytona：让任务继续运行 / 90 – 110 · 20 分钟

07 Agentic DevOps：先让测试变红 / 110 – 123 · 13 分钟

08 Agentic DevOps：从绿灯到发布 / 123 – 135 · 12 分钟

09 Lambda + vLLM：把模型服务接进来 / 135 – 155 · 20 分钟

10 Token 治理：谁可以花多少 / 155 – 165 · 10 分钟

11 模型网关与生态选型 / 165 – 175 · 10 分钟

12 交付一套可以维护的系统 / 175 – 180 · 5 分钟

80 – 90 分钟休息。课前完成账号配置、模型下载和沙箱预热。

# 01 从一张工单看完整系统

00 – 10 · 10 分钟

用户提交“取消订阅后仍收到扣费通知”。业务 Agent 查阅资料、决定下一步工具、保存带依据的回复草稿。Code Agent 负责构建和维护这个系统。两者的身份、工具和运行环境分开。

浏览器 → Vercel → Supabase 任务 → Render Agent → 模型服务 → 回复草稿

## 操作路径

- 验收：页面可访问、任务不丢、回复只属于当前用户。
- 课堂只生成草稿；真实发送和退款不是本次交付。
- 把运行证据写下来：URL、commit、CI run、job id 和模型版本。

## 操作示例

```
npm ci
npm test
npm run test:mutation
```

## 给 Code Agent 的任务

阅读 README 和 src/agent.mjs。解释两个 Agent 的职责，列出每条结论对应的文件位置；不要改文件。

## 验证成功的证据

让学员指出：哪一步需要模型判断，哪一步必须是确定性代码？

## 边界与常见误区

成功返回 HTTP 200，只证明接口返回了响应，不证明业务任务已经完成。

## 02 让 Code Agent 获得平台能力

10 – 25 · 15 分钟

把接入看成能力合同：入口是 MCP、CLI 或

API；身份来自项目账号；权限限定到课堂项目；每次操作都要有结果验证。先读后写，先确认目标 project id，再创建资源。

任务指令 → 平台工具 → 项目身份 → 操作结果 → 独立验证

### 操作路径

- GitHub：gh auth status；仓库、分支、PR、checks。
- Supabase：连接器管理新项目；CLI 管理本地数据库。
- Vercel / Render：项目配置、环境变量、部署状态。
- Lambda / Daytona：实例或沙箱 id、运行时间、清理责任。

### 操作示例

```
gh auth status
gh repo view mrvhao/agentist-supportops
gh run list --repo mrvhao/agentist-supportops
```

### 给 Code Agent 的任务

只读检查六个平台连接。仅返回账号、项目和权限是否可用，不输出任何 key。写出需要我补充的配置。

### 验证成功的证据

检查环境变量名与目标项目，不将密钥写进 prompt、截图、Git 或课程 PDF。

### 边界与常见误区

CLI 已登录不等于有该项目权限。403 先核对身份与权限，不尝试旁路凭证。

## 03 GitHub 与 Supabase：先让数据有家

25 – 45 · 20 分钟

工单提交在一个数据库事务中同时创建 ticket 和 job。用户 API 使用用户 JWT；后台 worker 使用专用服务端身份。个人工作区用 owner\_id 隔离，RLS 拦住跨用户读取。

Auth 用户 → submit\_ticket 事务 → tickets + jobs → 用户仅可读自己的行

### 操作路径

- 新建独立 Supabase 项目；不要在现有业务库演示迁移。
- 查看 migration 中的 RLS、grant 与 service-only RPC。
- 启动本地 Supabase，运行与 CI 完全相同的集成测试。
- 同时提交两次 claim，只允许一个 worker 获得同一任务。

### 操作示例

```
supabase start
node scripts/test-local-db.mjs
# 本地数据库可重建；线上迁移必须先核对 project ref
```

### 给 Code Agent 的任务

为“用户 B 不能读取用户 A 的工单”写集成测试。通过真实 Auth 登录获得两个 JWT，不使用 service role 模拟用户访问。

### 验证成功的证据

读取 integration/db.test.mjs 的断言，说明它在哪一层证明隔离。

### 边界与常见误区

Service role 可以绕过 RLS。后台必须继续按 job 中的 owner\_id 查询，不能信模型提供的用户编号。

## 04 Vercel：从仓库到可访问应用

45 – 60 · 15 分钟

网站承接登录、提交和查看结果，短请求提交后立即返回任务编号。耗时的 Agent 不放在一次网页请求里等待。部署成功后按真实用户路径检查，不能只看构建绿色。

Git commit → Vercel build → Preview URL → 登录 → 提交工单 → 轮询结果

### 操作路径

- 导入独立 GitHub 仓库并选择 public 输出目录。
- 配置 SUPABASE\_URL 和 SUPABASE\_ANON\_KEY；前端不需要 service role。
- 检查 /api/config 与 /api/tickets，确认没有暴露服务端密钥。
- 预览环境与生产环境分别配置，记录部署对应的 commit。

### 操作示例

```
vercel login
vercel link
vercel deploy
# 先验收 preview；生产发布是另一项明确操作
```

### 给 Code Agent 的任务

部署当前 commit 的预览版本。检查登录、工单提交、错误提示、窄屏布局，并给出预览 URL 与构建状态。

### 验证成功的证据

用户退出后页面清空；未登录请求必须返回 401；工单提交后返回任务 id。

### 边界与常见误区

预览链接可打开不等于后台 Agent 在线。数据库任务可能一直 queued，要继续看 Render。

## 05 Cloudflare 与支付边界

60 – 80 · 20 分钟

Cloudflare 在本案例中连接两种入口：用户访问网站的域名，以及 Render 访问 GPU 推理服务的受控入口。域名、TLS、Tunnel 和访问策略分别验证。Stripe 作为可选扩展，用测试模式事件补充订阅表。

网站域名 → Vercel | 推理域名 → Access → Tunnel → GPU localhost:8000

### 操作路径

- 只新增课堂子域名，不修改现有站点根域名或 DNS。
- cloudflared 在 GPU 主机建立出站连接；本地模型端口不裸露。
- Access Service Token 只给后台调用方；模型自身仍校验 API key。
- 支付扩展：验签 → event\_id 去重 → 更新权益；不把 success 页面当付款证据。

### 操作示例

```
# GPU 主机上，以已创建的配置运行
cloudflared tunnel --config /etc/cloudflared/config.yml run
# 模板见 deploy/cloudflared.example.yml
```

### 给 Code Agent 的任务

画出浏览器到网站、worker 到模型两条路径。指出每条路径的身份检查在哪里；不要更改现有 DNS。

### 验证成功的证据

无服务凭证的推理请求被拒绝；持有正确凭证的 /v1/models 请求成功。

### 边界与常见误区

本仓库未接 Stripe 实际支付。该扩展是设计练习，不在实测交付中冒充已完成。

## 06 Render 与 Daytona：让任务继续运行

90 – 110 · 20 分钟

Render worker 不依赖浏览器保持打开。任务状态存进 Supabase，进程重启后可以重新领取过期租约。Daytona 是 Code Agent 的隔离工作区，用于运行测试和复现，不承担持久业务队列。

queued → running + lease → completed | timeout → queued / failed / blocked

### 操作路径

- Render 从 render.yaml 配置后台 worker 和服务端环境变量。
- claim\_job 使用 FOR UPDATE SKIP LOCKED；lease\_token 拦住过期 worker 写回。
- 最多三次任务尝试；模型调用还有独立的轮数与超时限制。
- Daytona 只注入本次需要的变量，测试完成后停止或删除本次沙箱。

### 操作示例

```
npm run worker
# Render: build = npm ci; start = npm run worker
# 学员本地或 Daytona 内：npm test
```

### 给 Code Agent 的任务

为进程中断写回归测试：让租约过期，重新领取后旧 worker 不得完成任务。指出数据库与代码各自提供什么保证。

### 验证成功的证据

关网页不影响 worker；两个 worker 不重复领取；旧租约完成请求必须失败。

### 边界与常见误区

租约解决任务所有权，不提供任意外部副作用的 exactly-once。回复依靠唯一键与事务幂等。

## 07 Agentic DevOps：先让测试变红

110 – 123 · 13 分钟

给 Code Agent 一份验收合同，再让它写测试。课堂 PR 故意移除“草稿必须有依据”的检查，GitHub Actions 应在 PR 上显示失败。测试的职责是发现错误，不是替现有实现背书。

验收合同 → Code Agent 测试 → 故意回归 → PR → Actions 红灯

### 操作路径

- 阅读 tests/agent.test.mjs 的 missing evidence 用例。
- 打开 PR 的 Checks 页，定位实际失败断言。
- 区分单元测试、数据库集成测试、模型实测。
- 检查 test:mutation：移除证据闸后，对应断言真的失败。

### 操作示例

```
gh pr checks --repo mrvhao/agentist-supportops  
gh run view RUN_ID --repo mrvhao/agentist-supportops --log-failed
```

### 给 Code Agent 的任务

读取失败日志，只修复造成失败的业务代码。保留验收断言，不删除测试、不降低阈值、不把失败测试标记 skip。

### 验证成功的证据

失败必须由目标断言触发，不能因为依赖没装、网络失败或语法错误。

### 边界与常见误区

一个会一直绿的测试没有证明价值。我们需要知道改坏哪一行会让它变红。

## 08 Agentic DevOps：从绿灯到发布

123 – 135 · 12 分钟

Code Agent 修复后推送同一 PR，pull\_request 的 synchronize 事件启动下一轮测试。required checks 把测试结果变成合并条件。预览部署用于验收；合并后的部署仍要检查健康状态与业务闭环。

修复提交 → PR 再测 → required checks → 人工审阅 → 部署 → smoke test

### 操作路径

- unit-and-mutation：语法、单元与变异检测。
- database-and-worker：临时 Supabase，真实 SQL 与 worker 流程。
- PR 不接收生产 secrets；需要真实模型的验证走受控路径。
- 数据迁移采用向后兼容策略；回退版本不能撤销已经执行的外部动作。

### 操作示例

```
gh pr checks --repo mrvhao/agentist-supportops
# required checks:
# unit-and-mutation
# database-and-worker
```

### 给 Code Agent 的任务

为新 PR 配置自动测试触发器。说明创建 PR、追加 commit、重新打开 PR 时会跑什么，并验证分支规则真的阻止失败合并。

### 验证成功的证据

对比红绿两次 run 的 commit 和断言；绿色后仍需实际提交一张工单。

### 边界与常见误区

GitHub 私有仓库的分支保护可能受账号套餐限制；检查 API 结果，不把“workflow 已写”当作“门禁已生效”。

## 09 Lambda + vLLM：把模型服务接进来

135 – 155 · 20 分钟

先跑通应用，再更换推理来源。Lambda 提供 GPU 主机，vLLM 提供模型 HTTP 服务，两者不是同一种产品。本课用固定模型别名 support-agent，让应用配置与底层模型路径分离。

Lambda GPU → vLLM serve → /v1/chat/completions → 模型适配器 → Agent

### 操作路径

- 记录 GPU 类型、小时价、驱动、vLLM 与模型版本。
- 在隔离 Python 环境安装兼容 vLLM，预先下载模型。
- 先在 GPU 本机验证，再验证 Tunnel，最后接入 Render。
- 课堂之后释放本次实例；保存结果与版本，不依赖机器永远开着。

### 操作示例

```
export MODEL_PATH=Qwen/Qwen2.5-7B-Instruct
bash deploy/vllm.sh
# API key 从主机环境读取，不写入命令示例或 Git
```

### 给 Code Agent 的任务

检查 GPU 与模型显存需求，提出最小可运行部署。先验证 /v1/models，再跑固定工单集，报告输出结构、耗时与 usage。

### 验证成功的证据

模型替身测试通过不代表 vLLM 通过；真实返回必须单独保存模型名称与用量证据。

### 边界与常见误区

vLLM 本次部署目标是 NVIDIA GPU。Ollama 可作为个人电脑体验入口，SGLang 是可选服务引擎；不据单次演示排名。

# 10 Token 治理：谁可以花多少

155 – 165 · 10 分钟

治理位于模型调用之前。每次尝试先原子预留额度，再请求上游；成功后按 usage 结算。超时仍可能已经消费，所以将预留记为 unknown，不能退还后免费无限重试。

租户 → reserve 原子预算 → 模型调用 → measured / unknown → ledger

## 操作路径

- 预算按工作区和日期划分；并发预留在数据库中完成。
- 输入长度、输出上限、工具轮数、任务重试分别设置。
- 用两次并发 60 Token 预留测试 100 Token 上限，只能成功一次。
- 成本字段按配置单价估算；GPU 租金另计，不把零价误认为免费。

## 操作示例

```
npm test
# 核心：src/model.mjs
# SQL：reserve_usage / settle_usage
```

## 给 Code Agent 的任务

写测试证明额度不足时上游 fetch 从未执行；再模拟超时，证明预算没有被静默退回。

## 验证成功的证据

重复 settle 不重复扣费；上游失败进入 unknown，账本和预算仍能对上。

## 边界与常见误区

本课预算是保守 Token admission，不是逐 Token 截断或财务账单；实际 tokenizer 与上游 usage 需要核对。

# 11 模型网关与生态选型

165 – 175 · 10 分钟

模型适配器先统一应用接口，再决定是否引入

LiteLLM。增加备用模型时，分别验证协议、输出质量和数据许可。相同厂商的多个 key 不意味着不同故障域。

应用模型别名 → 路由策略 → 自托管 / 允许的外部 API → 统一用量

## 操作路径

- 部署模板见 `deploy/litellm.example.yaml`；它是扩展配置，不是假装已部署的网关。
- 故障转移只用于允许外发的课堂合成数据。
- 租卡：按存活时长收费；API：按输入输出收费；关注利用率与工程维护。
- 选型比较延迟、质量、隐私、迁移成本；六家可观测与多模态工具列为课后阅读。

## 操作示例

```
# 应用保持 MODEL_NAME=support-agent
# 只调整 MODEL_BASE_URL 与 MODEL_API_KEY
# 每次换供应商重新跑受控模型集成测试
```

## 给 Code Agent 的任务

给出两种流量下的选型：低频个人工具和全天稳定客服。写清输入输出量、GPU 利用率与维护成本假设。

## 验证成功的证据

拔掉主模型时，禁止外发的工单应延后或转人工；不能为可用性悄悄改变数据边界。

## 边界与常见误区

厂商价格随时间变化。报价必须包含日期、区域、硬件与计费单位，不照搬历史课程数字。

## 12 交付一套可以维护的系统

175 – 180 · 5 分钟

课程的最终产物不是一张架构图，而是一组可以检查的运行证据。学员选择一个小改动，要求 Code Agent 补测试、创建 PR、读结果、完成修复并验收部署。

应用 URL + PR + CI run + job result + usage ledger + cleanup record

### 操作路径

- 基础作业：新增一条帮助文档，补正常与无依据测试。
- 进阶作业：改重试策略，证明不会产生重复回复。
- 部署作业：接入自己的模型地址，运行固定工单集。
- 运维作业：记录资源 id、负责人、预算和停止方法。

### 操作示例

```
npm run check
npm test
npm run test:mutation
# 完整数据库测试在 PR 的 Actions 中可复核
```

### 给 Code Agent 的任务

提交变更前给出测试报告与剩余风险。只在所有必须检查通过后请求审阅；不要用“应该可以”替代运行证据。

### 验证成功的证据

三份材料同一章号：讲义解释，slides 带节奏，白板承载结构和课堂讨论。

### 边界与常见误区

未实测、待账号接入、模拟数据、真实测量必须分开标注。

# 复现与验收

代码仓库：[github.com/mrvhao/agentist-supportops](https://github.com/mrvhao/agentist-supportops)

本地：`npm ci → npm run check → npm test → npm run test:mutation`。数据库集成：`supabase start → node scripts/test-local-db.mjs`。

## 必过检查

unit-and-mutation：代码检查、行为测试、变异检测。database-and-worker：真实 Auth、RLS、并发领取、过期租约、幂等完成、并发预算和重复结算。

## 发布前

核对 commit、预览链接、环境变量、数据库迁移与用户隔离。真实模型 smoke test 单独记录，不能由替身测试替代。

## 资源清理

Lambda：只释放本次创建的实例 id。Daytona：只删除本次测试沙箱。Render 与 Supabase：长期保留需明确负责人和预算。

## 参考文档

### GitHub PR 与必需检查

<https://docs.github.com/en/pull-requests/reference/status-checks>

### Render 部署

<https://render.com/docs/deloys>

### Supabase API key 与权限

<https://supabase.com/docs/guides/getting-started/api-keys>

### vLLM 兼容接口

[https://docs.vllm.ai/en/latest/serving/online\\_serving/openai\\_compatible\\_server/](https://docs.vllm.ai/en/latest/serving/online_serving/openai_compatible_server/)

### Cloudflare Tunnel

<https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>